

FASE 1 – LOGIKA PENYELESAIAN MASALAH

Nama/NIM: Muhammad Navis Idris/ 26/585523/TK/66765

Kelompok: Mariposite

Problem 1 – The Last Astronaut

Pseudocode

```
START
INPUT N, K
IF K < 2 THEN K = 2

Buat array astronaut berisi 1, 2, 3, ..., N
jumlah = N
posisi = 0

WHILE jumlah > 1:
    indeks = (posisi + K - 1) MOD jumlah
    tereliminasi = astronaut[indeks]
    CATAT tereliminasi

    IF tereliminasi MOD 2 = 0 THEN
        K = K + 2
    ELSE
        K = K - 1
    IF K < 2 THEN K = 2
    ENDIF

    Hapus astronaut[indeks]
    jumlah = jumlah - 1
    posisi = indeks MOD jumlah

OUTPUT seluruh urutan eliminasi
OUTPUT astronaut yang tersisa
END
```

Penjelasan Logika

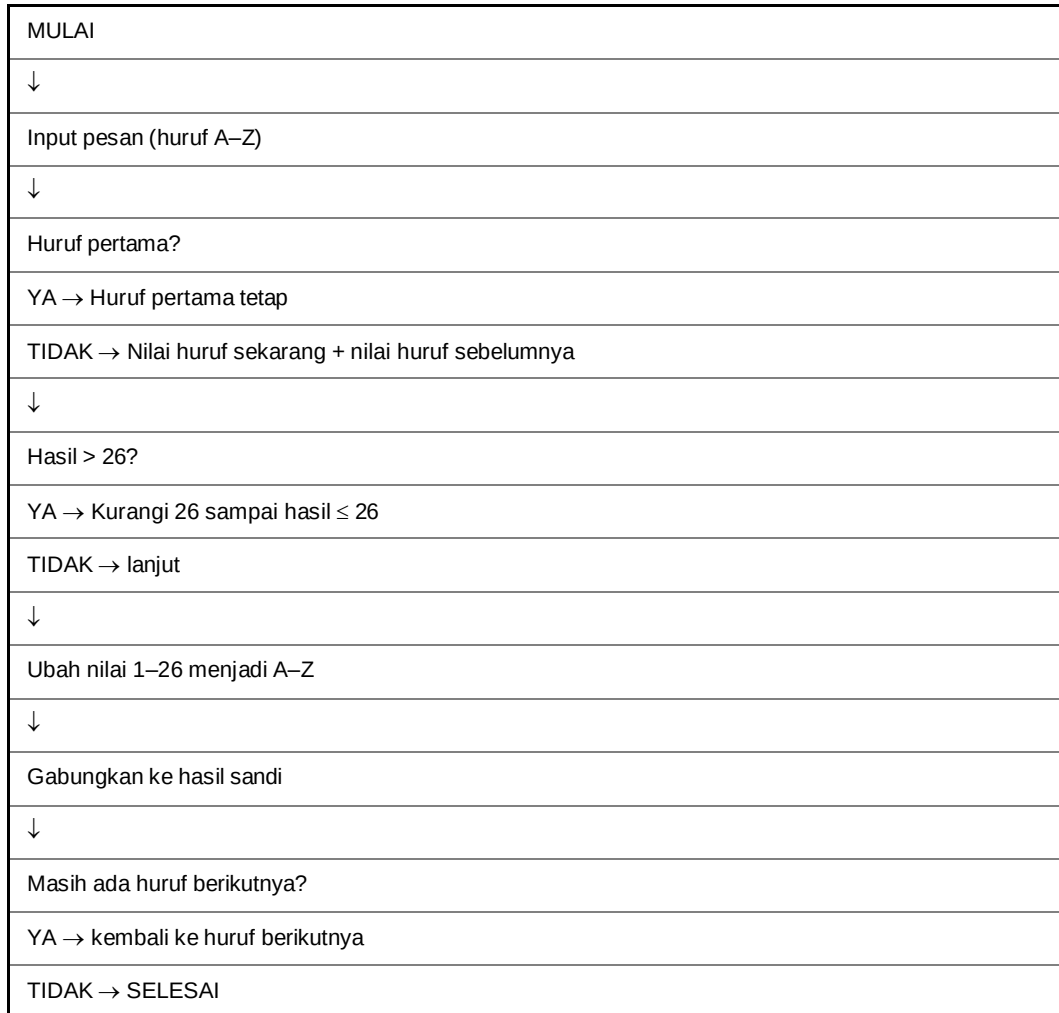
Problem *The Last Astronaut* merupakan proses eliminasi berbentuk lingkaran. Astronaut diberi nomor dari 1 sampai N dan proses dimulai dari astronaut nomor 1. Untuk setiap putaran, posisi astronaut yang dieliminasi ditentukan dengan rumus **(posisi + K - 1) mod jumlah**. Rumus tersebut membuat perhitungan tetap berjalan secara melingkar sehingga ketika mencapai astronaut terakhir, proses kembali ke astronaut pertama yang masih tersisa. Setelah seorang astronaut dihapus, perhitungan berikutnya dimulai dari astronaut yang berada tepat setelah astronaut yang dieliminasi.

Hal yang membedakan masalah ini dari eliminasi biasa adalah nilai K dapat berubah setelah setiap eliminasi. Jika nomor astronaut yang dieliminasi genap, K bertambah 2. Jika nomor tersebut ganjil, K berkurang 1. Namun, K tidak boleh kurang dari 2 sehingga jika hasil pengurangan membuat K di bawah 2, nilainya dikembalikan menjadi 2. Array digunakan untuk menyimpan astronaut yang masih bertahan; ketika satu astronaut dieliminasi, elemen setelahnya digeser untuk menutup posisi yang kosong. Proses diulang sampai hanya satu astronaut tersisa, dan nomor astronaut tersebut menjadi hasil akhir.

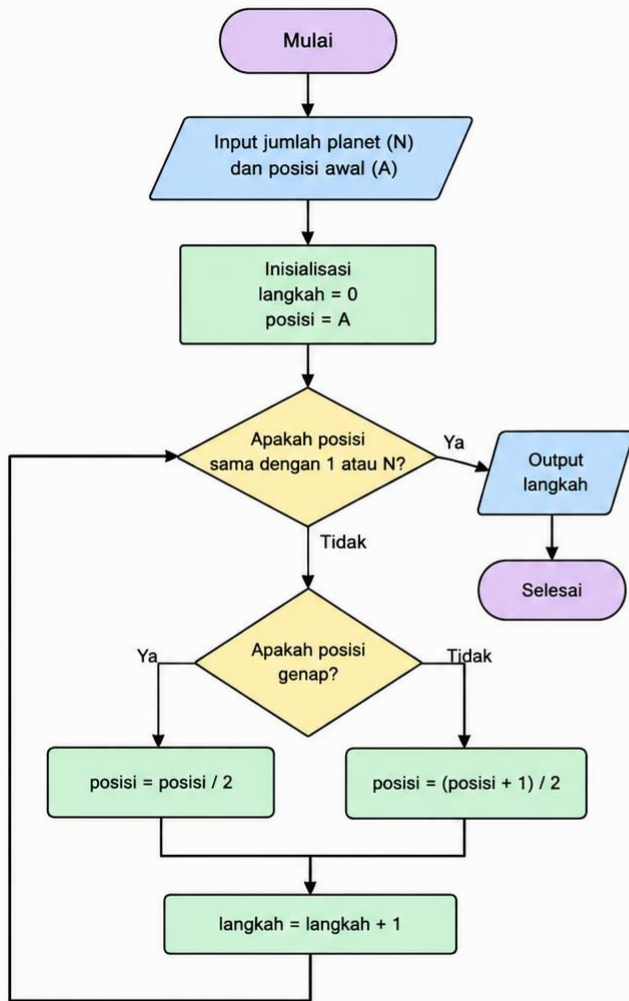
Problem 2 – Alien-In-The-Middle

Flowchart

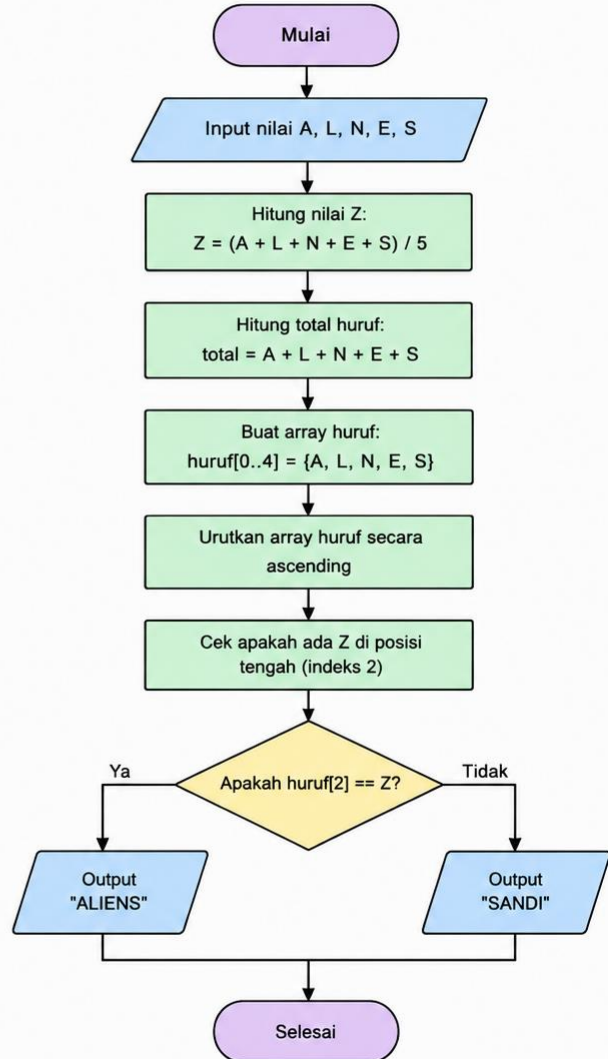
Alur: Mulai → input pesan → cek huruf pertama → jika ya, huruf tetap → jika tidak, jumlahkan nilai huruf sekarang dengan nilai huruf sebelumnya → jika hasil > 26, kurangi 26 hingga berada pada 1–26 → ubah angka menjadi huruf A–Z → masukkan ke hasil sandi → cek apakah masih ada huruf → ulangi sampai semua huruf diproses → selesai.



Problem 1: The Last Astronaut Flowchart



Problem 2: Alien-In-The-Middle Flowchart



Penjelasan Logika

Problem *Alien-In-The-Middle* menggunakan sistem sandi yang menentukan huruf baru berdasarkan huruf yang sedang diproses dan huruf tepat sebelumnya. Setiap huruf diberi nilai A=1, B=2, C=3, hingga Z=26. Huruf pertama tidak mengalami perubahan sehingga langsung dimasukkan ke hasil. Untuk setiap huruf berikutnya, nilai huruf tersebut dijumlahkan dengan nilai huruf sebelumnya pada pesan asli. Hasil penjumlahan kemudian diterjemahkan kembali menjadi huruf.

Karena hasil penjumlahan dapat lebih besar dari 26, sistem menggunakan perulangan untuk mengembalikannya ke rentang 1 sampai 26 dengan cara mengurangi 26. Dengan aturan ini, pergeseran setelah Z akan kembali ke A. Contohnya, pesan **ALIENS** menghasilkan **AMUNSG**: A tetap A, L+ A menghasilkan M, I+L menghasilkan U, E+I menghasilkan N, N+E menghasilkan S, dan S+N menghasilkan G setelah hasil 33 dikurangi 26. Proses terus dilakukan sampai seluruh karakter pesan selesai diproses.

Lampiran – Implementasi C++ Fase 2

Program berikut menggunakan hanya **iostream** dan tidak memakai fungsi dari library lain. Operasi yang diperlukan dibuat langsung di dalam program.

```
#include <iostream>
using namespace std;

// Problem 1: The Last Astronaut
void problem1() {
    int n, k;
    cout << "Masukkan N: ";
    cin >> n;
    cout << "Masukkan K: ";
    cin >> k;

    if (n <= 0) {
        cout << "N harus lebih dari 0.\n";
        return;
    }
    if (k < 2) k = 2;

    int *astronaut = new int[n];
    for (int i = 0; i < n; i++) {
        astronaut[i] = i + 1;
    }

    int jumlah = n;
    int posisi = 0;

    cout << "\nUrutan eliminasi: ";

    while (jumlah > 1) {
        int indeks = (posisi + k - 1) % jumlah;
        int tereliminasi = astronaut[indeks];

        cout << tereliminasi;
        if (jumlah > 2) cout << " -> ";

        // Aturan perubahan K berdasarkan nomor astronaut yang dieliminasi.
        if (tereliminasi % 2 == 0)
            k += 2;
        else {
            k -= 1;
            if (k < 2) k = 2;
        }

        // Menghapus elemen dengan menggeser elemen setelahnya ke kiri.
        for (int i = indeks; i < jumlah - 1; i++) {
            astronaut[i] = astronaut[i + 1];
        }

        jumlah--;

        // Setelah penghapusan, hitungan berikutnya dimulai dari
        // astronaut yang tepat setelah posisi astronaut yang dihapus.
        if (jumlah > 0)
            posisi = indeks % jumlah;
    }

    cout << "\nAstronaut terakhir yang bertahan: " << astronaut[0] << "\n";

    delete[] astronaut;
}

// Problem 2: Alien-In-The-Middle
```

```

void problem2() {
char pesan[1000];
cout << "\nMasukkan pesan (huruf A-Z saja): ";
cin >> pesan;

cout << "Hasil sandi: ";

int i = 0;
while (pesan[i] != '\0') {
char hasil;

if (i == 0) {
// Huruf pertama tidak berubah.
hasil = pesan[i];
} else {
int nilaiSekarang = pesan[i] - 'A' + 1;
int nilaiSebelumnya = pesan[i - 1] - 'A' + 1;
int nilaiBaru = nilaiSekarang + nilaiSebelumnya;

// Jika melewati Z, kembali dari A.
while (nilaiBaru > 26)
nilaiBaru -= 26;

hasil = char('A' + nilaiBaru - 1);
}

cout << hasil;
i++;
}

cout << "\n";
}

int main() {
int pilihan;

cout << "=== TANTANGAN FASE 2 ===\n";
cout << "1. The Last Astronaut\n";
cout << "2. Alien-In-The-Middle\n";
cout << "Pilih problem: ";
cin >> pilihan;

if (pilihan == 1)
problem1();
else if (pilihan == 2)
problem2();
else
cout << "Pilihan tidak valid.\n";

return 0;
}

```